ZZedc Permissions Hierarchy via Google Sheets: Design, Analysis, and Construction

2026-05-28 18:33 PDT

This document covers two related topics. The first is the design and implementation of the Google Sheets change monitor added to ZZedc for the Prazosin Sleep Study (Large), protocol PRAZ-L-2026. The second is an analysis of standard EDC permission hierarchies and a construction guide for building the correct hierarchy in ZZedc using the Google Sheets seed-import pathway.

Each topic is treated in two registers: a technical section using database and systems terminology, followed by a plain-language section for the data scientist and study staff who will operate the system.

Part 1: Change Monitor – Technical Description (Database Expert Register)

Background and motivation

ZZedc supports a Google Sheets workbook as a bootstrap seed source for three configuration relations: the principal registry (`Study_Users`), the field metadata catalogue (`Data_Dictionary`), and the constraint definition table (`validation_rules`). In the multi-site prazosin trial, each site coordinator holds editor-level ACL grants on the `Study_Users` tab of their per-site workbook. The `Data_Dictionary` and `validation_rules` tabs are viewer-only for coordinators, enforced via Google Sheets tab protection. This arrangement allows non-technical site staff to propose changes to their site's user roster without requiring data-scientist mediation for every routine addition.

Without a change-detection layer, the only path from sheet to database was a manual, full-table re-import via `setup_zzedc_from_gsheets()`. That operation writes directly to the live `edc_users` relation with no review gate. A coordinator could add an account with an incorrect role, or modify another user's site scope, and the change would land in the database on the next import.

The modification described here introduces a review gate between the external mutation source (the sheet) and the operational principal registry.

Design constraint: non-destructive staging

The central constraint is that a proposed roster change must not activate until a credentialled reviewer has approved it. The existing import path presented a structural obstacle: `db_insert_user()` is an INSERT-only operation; modifications require raw `UPDATE` SQL. Neither path supports a pending state. Staging proposals in a separate relation (`user_proposals`) solves this without touching `edc_users` or requiring a schema migration to the existing user table.

An analogous staging relation (`dsl_rule_proposals`) was implemented for data-scientist-initiated validation rule proposals. The two staging relations are independent; their approval functions operate on separate queues.

Schema: `user_proposals`

```
CREATE TABLE IF NOT EXISTS user_proposals (
  id                INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
  proposal_id        TEXT UNIQUE NOT NULL,  -- username + compacted timestamp
  username           TEXT NOT NULL,
  change_type        TEXT NOT NULL,         -- 'NEW' or 'MODIFIED'
  proposed_full_name TEXT,
  proposed_email     TEXT,
  proposed_role      TEXT,
  proposed_site_id   TEXT,
  proposed_active    INTEGER,
  current_role       TEXT,                  -- snapshot at staging; NULL for NEW
  current_email      TEXT,
  sheet_id           TEXT NOT NULL,
  staged_by          TEXT NOT NULL,
  staged_at          TEXT NOT NULL,
  status             TEXT NOT NULL DEFAULT 'PENDING',
  reviewed_by        TEXT,
  reviewed_at        TEXT,
  review_comments    TEXT
);
```

Schema: `dsl_rule_proposals`

```
CREATE TABLE IF NOT EXISTS dsl_rule_proposals (
  id              INTEGER PRIMARY KEY AUTOINCREMENT,
  proposal_id     TEXT UNIQUE NOT NULL,  -- rule_id + compacted timestamp
  rule_id         TEXT NOT NULL,
  change_type     TEXT NOT NULL,         -- 'NEW', 'MODIFIED', 'DELETED'
  proposed_dsl    TEXT,
  current_dsl     TEXT,                  -- snapshot at staging; NULL for NEW
  field_code      TEXT,
  form_code       TEXT,
  rule_name       TEXT,
  error_message   TEXT,
  severity        TEXT,
  rule_category   TEXT,
  sheet_id        TEXT NOT NULL,
  staged_by       TEXT NOT NULL,
  staged_at       TEXT NOT NULL,
  status          TEXT NOT NULL DEFAULT 'PENDING',
  reviewed_by     TEXT,
  reviewed_at     TEXT,
  review_comments TEXT
);
```

Change detection

`diff_gsheets_users()` performs a read-side diff against `edc_users`:

1. Fetches the `Study_Users` tab via `googlesheets4::read_sheet()`.
2. Queries `SELECT username, role, email FROM edc_users`.

3. Classifies each sheet row as `NEW` (username absent from DB) or `MODIFIED` (username present but role or email differs). Deletions are flagged in log output but never auto-staged; removing a principal from a live trial database requires explicit data-scientist action.

The analogous `diff_gsheets_rules()` diffs the `validation_rules` tab against `dsl_validation_rules` using DSL content equality.

Staging and idempotency

Both staging functions (`stage_gsheets_user_proposals()`, `stage_gsheets_proposals()`) are idempotent with respect to content. If a `PENDING` proposal already exists for the same key (username or rule_id) with identical proposed values, the row is not duplicated. If the proposed values have changed since the last poll (a coordinator revised their edit before the manager reviewed it), the existing `PENDING` row is updated in place. This prevents proposal accumulation across poll intervals while ensuring the queue always reflects the coordinator's current intent.

Approval and database rebuild

`approve_user_proposals()` for each approved `proposal_id`:

1. Looks up the reviewer's role in `edc_users` and verifies it is `admin`, `pi`, or `studymanager`. Returns an `auth` error immediately if not; no database writes occur.
2. Validates `status = 'PENDING'`.
3. For `NEW`: calls `db_insert_user()` with the proposed values and a cryptographically random initial password (12-character alphanumeric, hashed with the system salt). The account holder must change this password on first login.
4. For `MODIFIED`: issues an `UPDATE` to `edc_users` using `COALESCE` to apply only the fields present in the proposal.
5. Marks the proposal `'APPROVED'` and records `reviewed_by`, `reviewed_at`, and any comments.

`reject_user_proposals()` applies the same role check before marking the proposal `'REJECTED'`.

`approve_proposals()` (for DSL rules) additionally recompiles the DSL expression to R and SQL validator expressions before writing to `dsl_validation_rules`.

Both reject functions mark the proposal `'REJECTED'` and leave the live relation untouched.

Notification architecture

The notifier is a dependency-injected function passed as the `notifier` argument of `poll_gsheets_and_stage()`. Signature: `function(proposals_df)`. The default implementation (`default_proposal_notifier()`) appends a plain-text summary to a log file resolved from `ZZEDC_PROPOSAL_LOG`. A production deployment substitutes a `blastula`-based SMTP function with no changes to the detection and staging logic.

Cron integration and multi-workbook polling

Because each site has its own workbook, the poll script iterates over four sheet IDs, one per site, resolved from environment variables (`PRAZ_SHEET_UCSD`, `PRAZ_SHEET_UCLA`, `PRAZ_SHEET_UCSF`, `PRAZ_SHEET_STANFORD`). The function returns `NULL` silently when no changes are detected, producing no log output on no-op runs.

Audit trail

All state transitions – staging, approval, rejection – are recorded. User proposal transitions are recorded in `user_proposals` itself (the `reviewed_by`, `reviewed_at`, `review_comments` columns). DSL rule transitions additionally write to `dsl_rule_change_log` via `log_dsl_rule_action()`, which captures old and new DSL values and the reviewer's identity.

Part 2: Permissions Hierarchy – Technical Analysis (Database Expert Register)

Standard tier structure for a multi-site clinical EDC

A well-governed EDC for a project of this size (200 subjects, 4 sites) conventionally defines four tiers.

Tier 0: System administrator (technical root) Operates below the application layer. Has filesystem access to the `.db` file and can bypass all application-layer permission checks. Is not registered as an application user. Can create additional Tier 0 accounts. Is never used for day-to-day study operations. Corresponds to the OS account running the Shiny server process and the account that can `ssh` to the server.

Tier 1: Study administrator (functional root within the project) One or two named application accounts. Typically the PI and a backup. Capabilities: create or deactivate any user at any tier; grant roles up to but not exceeding their own (non-escalation invariant); modify the data dictionary and validation rules; approve any pending proposal; view all data across all sites. In REDCap terms: holds `design`, `user_rights`, `data_access_groups`, and full export rights.

Tier 2: Study manager (coordinating centre) Best practice distinguishes two sub-roles that may be merged into one account or kept separate depending on the trial's governance structure:

- *Configuration manager*: can author and submit changes to the data dictionary and validation rules (`design` in REDCap); can view (not write) the full user roster; cannot create accounts at Tier 1; cannot grant `user_rights` to others.
- *User administrator*: can create and deactivate accounts at Tier 3 and below; cannot modify the data dictionary or validation rules.

The key invariant: a Tier 2 account cannot promote another account to Tier 1. The non-escalation rule must be enforced at the application layer, not by convention.

Tier 3: Site coordinator Data entry for their assigned site only. Cannot view other sites' data. Can propose changes to their site's user roster (via Google Sheets). Cannot modify any configuration directly.

Tier 4 (read-only): Monitor, sponsor, auditor Read-only access. No write permissions of any kind.

REDCap's permission model at this project scale

REDCap separates permissions into two orthogonal axes: project-level rights and per-instrument access levels. The project-level rights most relevant to this analysis are:

Right	Scope
`design`	Modify instruments, data dictionary, branching logic, validation
`user_rights`	Add, modify, or remove project user accounts
`data_access_groups`	Create and assign Data Access Groups (site isolation)
`data_export_tool`	Export records; can be restricted to de-identified
`lock_record`	Lock and unlock completed records
`record_create`, `record_delete`	Record lifecycle management
`alerts`	Configure automated notifications

REDCap enforces non-escalation: a user with `user_rights` can only grant permissions up to (not exceeding) their own. The system administrator (`superadmin`) is a separate technical account outside all projects.

For a 4-site 200-subject trial, the standard REDCap account matrix is:

Account	`design`	`user_rights`	DAG	Export	Notes
Superadmin	system	system	–	–	Tier 0; not a project account
PI	yes	yes	all	full	Tier 1
CC data manager	yes	no	all	full	Tier 2 config
CC user admin	no	yes (Tier 3 only)	all	full	Tier 2 user admin
Site coordinator	no	no	own site	none or de-id	Tier 3
Monitor	no	no	all	de-id	Tier 4

ZZedc role model: current state (v0.6.2)

The `dsl_rule_permissions` table now defines six roles. The `studymanager` row was added in v0.6.2.

Role	can_view	can_create	can_edit	can_delete	can_approve	can_activate
`admin`	1	1	1	1	1	1
`pi`	1	0	0	0	1	1
`studymanager`	1	1	1	0	1	1
`data_manager`	1	1	1	0	0	1
`coordinator`	1	0	0	0	0	0
`monitor`	1	0	0	0	0	0

Gap analysis and resolution status

Gap 1 – No `studymanager` role. RESOLVED (v0.6.2) `setup_default_dsl_permissions()` now includes `studymanager` with full configuration-write and approval authority. `StudyManager` is also registered in `edc_roles` during `create_wizard_database()`. The exported `migrate_add_studymanager_role()` adds both entries to databases initialised before v0.6.2 without requiring a full re-import.

`approve_user_proposals()` and `reject_user_proposals()` both verify the reviewer's role against `edc_users` at call time, requiring `admin`, `pi`, or `studymanager`. A `data_manager` or `coordinator` account calling these functions receives an `auth` error before any proposal state is changed.

Gap 2 – No non-escalation enforcement. RESOLVED (v0.6.2) `user_management_module.R` now contains a `.ROLE_TIER` hierarchy and an `.assignable_roles()` helper. `user_management_server()` accepts an `actor_role` parameter populated by `admin_dashboard_server()` from the active session's role. The add-user and edit-user modals call `updateSelectInput()` with choices filtered to tiers strictly below the actor's own tier. A `studymanager` sees only `Coordinator` and `Monitor` in the role dropdown; an `admin` or `pi` sees all roles.

Gap 3 – `pi` cannot author rules directly. RETAINED BY DESIGN `can_create = 0` and `can_edit = 0` for the `pi` role remain unchanged. The PI can approve rules proposed by a `studymanager` or `data_manager` but cannot author them directly. This enforces separation of authoring and approval at the permission layer and is consistent with good-practice two-person controls for constraint changes in a regulated trial. A PI who genuinely needs authoring access can be given an additional `data_manager`-role account, or the `pi` row in `dsl_rule_permissions` can be updated via `migrate_add_studymanager_role()`-style SQL by the data scientist.

Gap 4 – Tier 0 / application admin conflation. RESOLVED (v0.6.2) The config template (`inst/templates/zzedc_config_template.yml`) and the prazosin-large technical guide (Section 5) now define a `pi:` section alongside `admin:`. When `pi_username` is present and distinct from `admin_username`, `create_wizard_database()` inserts a PI-role account in addition to the sysadmin account. The `admin` account is documented as a technical sysadmin reserved for system-level operations.

Part 3: Constructing the Permissions Hierarchy via Google Sheets

The Study_Users workbook as the authoritative seed source

The `Study_Users` tab is the principal registry seed: it defines every application user before the database is initialised. Getting the role assignments correct in the workbook before the first import is cheaper than correcting them after go-live, because post-import role changes require direct database writes or admin UI operations rather than a re-import.

As of v0.6.2, the `studymanager` role, non-escalation enforcement, and PI/sysadmin account separation are all built into the initialisation path. The construction process is:

1. Write `zzedc_config.yml` with separate `admin:` (sysadmin) and `pi:` sections.
2. Define the full desired hierarchy in the `Study_Users` tab of each workbook, using the account matrix below.

3. Run `zzedc::init(mode = 'config', config_file = 'zzedc_config.yml')`. This creates the sysadmin and PI accounts and registers all six roles including `studymanager` in `dsl_rule_permissions`.
4. Import the coordinating centre workbook, then each site workbook, with `setup_zzedc_from_gsheets()`.
5. Verify each account's effective permissions with a dry-run login.

For databases initialised before v0.6.2, run `migrate_add_studymanager_role(db_path = ...)` once to backfill the `studymanager` permission row and `edc_roles` entry without re-importing any data.

Recommended account matrix for PRAZ-L-2026

The following `Study_Users` tab structure is recommended. The coordinating centre workbook holds all non-coordinator accounts; each site workbook holds only that site's coordinator accounts.

Coordinating centre workbook (`Study_Users` tab):

username	full_name	role	site_id	active	Notes
`sysadmin`	System Administrator	Admin	ALL	TRUE	Tier 0/1 technical account; no study operations
`psmith`	Professor Smith	PI	ALL	TRUE	Tier 1; approves proposals, cross-site read
`cc_manager`	CC Study Manager	StudyManager	ALL	TRUE	Tier 2; dictionary + user approval authority
`cc_monitor`	CC Monitor	Monitor	ALL	TRUE	Tier 4; read-only

Per-site workbooks (`Study_Users` tab, one per site):

username	full_name	role	site_id	active	Notes
`ucsd_coord`	UCSD Coordinator	Coordinator	UCSD	TRUE	Tier 3
`ucla_coord`	UCLA Coordinator	Coordinator	UCLA	TRUE	Tier 3
`ucsf_coord`	UCSF Coordinator	UCSF	UCSF	TRUE	Tier 3
`stanford_coord`	Stanford Coordinator	Coordinator	Stanford	TRUE	Tier 3

Configuration file: separating sysadmin from PI

Write `zzedc_config.yml` with an `admin:` section for the technical account and a `pi:` section for the study PI. Both sections are parsed by `init()` and result in distinct database accounts.

```
admin:
  username: "sysadmin"
  fullname: "System Administrator"
  email: "sysadmin@university.edu"
  password: "ChangeToVaultManagedSecret!"

pi:
  username: "psmith"
  fullname: "Professor Smith"
  email: "psmith@university.edu"
  password: "TemporaryPI2026!"
```

The `sysadmin` account is used only for system initialisation, database migrations, and emergency recovery. Its password should be stored in an institutional secrets vault and rotated after initial setup. If the PI and system administrator are the same person, leave `pi:` empty; the admin account will function as both.

Migration for existing databases (pre-v0.6.2)

Databases initialised before v0.6.2 lack the `studymanager` row in `dsl_rule_permissions` and `edc_roles`. Run once:

```
library(zzedc)
migrate_add_studymanager_role(db_path = 'data/praz-large.db')
```

This is idempotent: safe to run on databases that already have the row. No data migration is required for the non-escalation or PI/sysadmin changes; those are application-layer and config-layer controls that take effect on the next application restart.

Part 4: Change Monitor – Plain-Language Description (Data Scientist Register)

What was the problem?

Coordinators at each recruitment site have editor access to the `Study_Users` tab of their site's Google Sheets workbook. This lets them propose new user accounts or request role changes for their site team. Before the change monitor was built, the only way to get those edits into the ZZedc database was for you to run a manual re-import. There was no checkpoint: the change went live immediately, whether or not it was correct.

For a running clinical trial, that is not safe. A coordinator could accidentally assign the wrong role to an account, or add a user with an incorrect site scope, and that account would have incorrect access from the moment you ran the import.

What was built

Three pieces were added:

1. A staging table in the database. When the monitor detects a coordinator's edit in the `Study_Users` sheet, it does not change the live user roster. It writes the proposed change to a separate holding table called `user_proposals`. Think of it as an inbox: proposals sit there until the manager acts on them.

2. A polling script and a notifier. A script runs on the study server every 15 minutes. It reads each site's workbook, compares it with the current user roster, and for any row that has changed, writes a proposal and triggers a notification. By default the notification is a plain-text log file. For production use you supply a short function that sends an email (a `blastula` example is in the technical guide, Section 7a.3).

3. Approval and rejection commands. When the manager receives the notification, they run:

- `get_pending_user_proposals()` to see the queue.
- `approve_user_proposals()` to accept one or more proposals. New accounts are created immediately with a temporary password; the account holder must change it on first login.
- `reject_user_proposals()` to decline a proposal with a recorded reason.

What you need to do to set this up

1. Write `zzedc_config.yml` with separate `admin:` and `pi:` sections as shown in Part 3. Run `zzedc::init(mode = 'config', ...)`. This creates both accounts and registers all six roles automatically.
2. Fill in the `Study_Users` tab of each workbook using the account matrix in Part 3.
3. Import the coordinating centre workbook first: `setup_zzedc_from_gsheets(sheet_url = CC_URL, db_path = ...)`.
4. Import each site workbook: `setup_zzedc_from_gsheets(sheet_url = SITE_URL, db_path = ...)`.
5. If upgrading an existing database (pre-v0.6.2), run `migrate_add_studymanager_role(db_path = ...)` once.
6. Create `poll_users.R` and add the cron entry per Section 7a.2 of the technical guide.
7. Set the four `PRAZ_SHEET_*` environment variables and the SMTP variables on the server.

What the standard permission hierarchy means for day-to-day operations

- **`sysadmin`**: used only by you (the data scientist) for initialisation and emergency operations. Never log in as `sysadmin` to enter or review study data.
- **`psmith` (PI)**: approves pending user and rule proposals; views cross-site data; does not routinely change database configuration.
- **`cc_manager` (study manager)**: maintains the data dictionary and validation rules; approves or rejects coordinator user proposals; cannot create admin or PI accounts.
- **Site coordinators**: data entry only; can propose new accounts for their site via the Google Sheet.
- If the PI wants to add a second study manager, add a new row to the coordinating centre `Study_Users` tab with `role = StudyManager` and put the proposal through the normal approval workflow.

Files modified or created

File	Change	Version
R/gsheets_monitor.R	10 exported functions: user and DSL rule proposal queues; role check in approve/reject	v0.6.0–0.6.2
R/validation_gsheets_integration.R	studymanager added to setup_default_dsl_permissions(); migrate_add_studymanager_role() exported	v0.6.2
R/setup_wizard_utils.R	StudyManager added to edc_roles; PI account creation from pi: config section	v0.6.2
R/user_management_module.R	.ROLE_TIER, .assignable_roles(), actor_role parameter, filtered role dropdowns	v0.6.2
R/admin_dashboard_module.R	Passes user_session$role as actor_role to user_management_server()	v0.6.2
R/init.R	Parses optional pi: config section	v0.6.2
inst/templates/zzedc_config_template.yml	Added pi: section; updated Example 2	v0.6.2
NAMESPACE	11 new export() entries across v0.6.0–0.6.2	v0.6.0–0.6.2
vignettes/prazosin-large/prazosin-large-technical-guide.Rmd	Sections 7.3, 7a, Section 5 config block updated	v0.6.0–0.6.2
vignettes/prazosin-large/prazosin-large-user-guide.Rmd	Section 9, troubleshooting, glossary	v0.6.0–0.6.1
docs/gsheets-monitor-design.md	This document	v0.6.0–0.6.2

Rendered on 2026-05-30 at 13:50 PDT. Source: ~/prj/sfw/05-zzedc/zzedc/docs/gsheets-monitor-design.md